

Program Drill — Set 4

Three full programs: a file upload handler, a pub/sub with replay, and a scheduler

October 2026

How to run it.

Same reading order (types, state, signatures, bodies, helpers, lifecycle, clauses). 35 minutes per program.

Program K — Upload handler with virus scan and storage

Requirement

1. `POST /upload` accepts one file ≤ 5 MB of type `image/png` or `image/jpeg`; else 400/413.
2. The file is scanned; if infected, respond 422 and delete the temp file.
3. Clean files are stored under a random name; respond `{ url }`.
4. The temp file is always removed, on every path, including errors.
5. The caller's user id (from the token) is recorded with the stored file.

```
1  import fs from "fs/promises";
2  import { randomUUID } from "crypto";
3  const MAX = 5 * 1024 * 1024;
4  const TYPES = ["image/png", "image/jpeg"];
5
6  app.post("/upload", upload.single("file"), async (req, res) => {
7    const f = req.file;
8    if (!f) return res.status(400).json({ error: "no file" });
9    if (f.size > MAX) return res.status(413).end();
10   if (!TYPES.includes(f.mimetype)) return res.status(400).json({ error: "type" });
11
12   const infected = await scan(f.path);
13   if (infected) {
14     res.status(422).json({ error: "infected" });
15     await fs.unlink(f.path);
16   }
17
18   const name = randomUUID() + path.extname(f.originalname);
19   await storage.put(name, await fs.readFile(f.path), { owner: req.body.userId });
20   await fs.unlink(f.path);
21   res.json({ url: `/files/${name}` });
22 });
```

Reading sequence

Step	Lines
types	none; <code>req.file</code> is multer's <code>File</code>
state	3–4
signature	6 (async handler after multer)
body	7–21
helpers	<code>scan</code> , <code>storage.put</code> (external)
lifecycle	none
clauses	

Notes

Line	Note
6	multer has already written the temp file before the handler runs; any early <code>return</code> (8–10) leaves it on disk
9	size over limit: returns 413 but does NOT unlink the temp file: clause 4
10	type check uses <code>mimetype</code> , which is client-supplied; a renamed <code>.exe</code> with <code>image/png</code> passes (sniff magic bytes for a real check)
10	returns without unlink: clause 4
12	awaited scan

Line	Note
13–16	infected: responds, unlinks, then FALLS THROUGH to line 18 and stores the infected file and sends a second response (no <code>return</code>)
18	extension from <code>originalname</code> , client-supplied: <code>../../x</code> is not possible via <code>extname</code> , but <code>.html</code> or <code>.svg</code> can be stored and served from <code>/files/</code> (stored-XSS vector)
19	<code>req.body.userId</code> : owner from the request body, attacker-controlled: clause 5 violated
19	if <code>storage.put</code> throws, line 20 never runs: temp file leaks: clause 4
20	unlink on the success path only

Clauses

Clause	Status
1 size and type	size yes; type by client header only
2 infected → 422 + delete	422 yes, delete yes, but falls through and stores it anyway
3 random name, <code>{ url }</code>	yes; extension is attacker-chosen
4 temp always removed	NO: early returns (8–10) and throws (12, 19) leak it
5 owner from token	NO: from body

Findings

- Line 16: missing `return` after the infected branch; the file is stored and a second response is attempted.
- Line 19: owner from `req.body.userId`; use the authenticated identity.
- Lines 9, 10, 12, 19: temp file not removed on early returns or thrown errors; wrap the body in `try/finally { await fs.unlink(f.path).catch(() => {}) }`.
- Line 10: `mimetype` is client-supplied; verify by content (magic bytes) or at least restrict the stored extension to the verified type.
- Line 18: extension taken from the client name; derive it from the verified type instead.

Positives:

awaited scan before storage; random storage name; size check before scan.

Confidence 1 • Reject.

Clause 5 is an auth hole; clause 2 stores the infected file.

Program L — Pub/sub with replay buffer

Requirement

1. `publish(topic, msg)` delivers to all current subscribers of the topic.
2. `subscribe(topic, fn, { replay: n })` first delivers the last `n` messages on that topic (oldest first), then live messages.
3. The buffer per topic keeps at most 100 messages.
4. `unsubscribe` returns from `subscribe`; after it, no further deliveries.
5. A subscriber that throws must not affect others; the error is reported via `onError`.

```
1  type Fn = (m: unknown) => void;
2  class PubSub {
3    private subs = new Map<string, Set<Fn>>();
4    private buf = new Map<string, unknown[]>();
5    constructor(private onError: (e: unknown) => void) {}
6
7    publish(topic: string, msg: unknown) {
8      const b = this.buf.get(topic) ?? [];
9      b.push(msg);
10     if (b.length > 100) b.shift();
11     this.buf.set(topic, b);
12     for (const fn of this.subs.get(topic) ?? []) {
13       try { fn(msg); } catch (e) { this.onError(e); }
14     }
15   }
16
17   subscribe(topic: string, fn: Fn, opts: { replay?: number } = {}): () => void {
18     const set = this.subs.get(topic) ?? new Set<Fn>();
19     set.add(fn);
20     this.subs.set(topic, set);
21     const n = opts.replay ?? 0;
22     const b = this.buf.get(topic) ?? [];
23     for (const m of b.slice(-n)) fn(m);
24     return () => set.delete(fn);
25   }
26 }
```

Reading sequence

Step	Lines
types	1
state	3–4
signatures	5, 7, 17
bodies	7–15, 17–25
helpers	none
lifecycle	none
clauses	

Notes

Line	Note
8–11	buffer append with cap: <code>shift</code> after exceeding 100 keeps 100: correct; <code>?? []</code> then <code>set</code> is the right idiom
12	iterates the live <code>Set</code> ; a subscriber that unsubscribes during delivery: deleting from a <code>Set</code> during <code>for...of</code> is safe in JS; the deleted one is skipped if not yet reached (acceptable)
13	per-subscriber try/catch: clause 5
18–20	<code>?? new Set()</code> then <code>set</code> : correct
21	<code>replay</code> defaults to 0
22–23	<code>b.slice(-n)</code> : with <code>n = 0</code> , <code>slice(-0)</code> is <code>slice(0)</code> → the ENTIRE buffer is replayed; clause 2 says 0 should replay nothing
23	replayed messages are delivered via <code>fn(m)</code> with NO try/catch: a throwing subscriber during replay propagates out of <code>subscribe</code> and the <code>onError</code> path is bypassed; clause 5

Line	Note
23	replay happens AFTER <code>set.add</code> (line 19): a <code>publish</code> cannot interleave (single-threaded, no await), so ordering is fine
24	closure over <code>set</code> : correct; double-unsubscribe is a no-op (<code>Set.delete</code>)

Clauses

Clause	Status
1 deliver to current subs	yes
2 replay last n, oldest first	wrong for n = 0 (replays all); order correct
3 cap 100	yes
4 unsubscribe	yes
5 isolate throwing subs	live yes; replay no

Findings

- Line 23: `slice(-0)` replays the whole buffer; guard `n > 0 ? b.slice(-n) : []`.
- Line 23: replay delivery lacks the try/catch of line 13; wrap and route to `onError`.
- Line 4: topics are user-supplied strings used as `Map` keys: fine (`Map`); say so.

Positives: `Map/Set` throughout; cap via `shift`; per-subscriber isolation on live delivery; idempotent unsubscribe.

Confidence 3 · Request changes.

Two defects on the replay path only.

Program M — Cron-like scheduler

Requirement

1. `schedule(name, everyMs, fn)` runs `fn` every `everyMs`; a run that is still in progress when the next tick arrives is skipped (no overlap).
2. `cancel(name)` stops it; cancelling an unknown name is a no-op.
3. `fn` errors are logged with the name and do not stop the schedule.
4. `shutdown()` cancels everything and resolves once no run is in progress.

```
1  type Entry = { timer: ReturnType<typeof setInterval>; running: boolean; fn: () =>
Promise<void> };
2  export class Scheduler {
3    private entries = new Map<string, Entry>();
4
5    schedule(name: string, everyMs: number, fn: () => Promise<void>) {
6      const entry: Entry = { running: false, fn, timer: undefined! };
7      entry.timer = setInterval(async () => {
8        if (entry.running) return;
9        entry.running = true;
10       try { await fn(); } catch (e) { console.error(name, e); }
11       entry.running = false;
12     }, everyMs);
13     this.entries.set(name, entry);
14   }
15
16   cancel(name: string) {
17     const e = this.entries.get(name);
18     if (!e) return;
19     clearInterval(e.timer);
20     this.entries.delete(name);
21   }
22
23   async shutdown() {
24     for (const name of this.entries.keys()) this.cancel(name);
25     while ([...this.entries.values()].some((e) => e.running)) await sleep(20);
26   }
27 }
```

Reading sequence

Step	Lines
types	1
state	3
signatures	5, 16, 23
bodies	5–14, 16–21, 23–26
helpers	none (<code>sleep</code> external)
lifecycle	7, 19, 24–25
clauses	

Notes

Line	Note
6	<code>timer: undefined!</code> to satisfy the type before assignment; assigned on line 7 synchronously; acceptable, say so
7	async callback inside <code>setInterval</code> : the interval ignores the returned Promise; errors are caught inside (10), so nothing is unhandled; fine
8–9	overlap guard: check then set, no await between: correct
10	<code>await fn()</code> in <code>try</code> : caught; logged with name: clause 3
11	reset; if <code>fn</code> throws, the <code>catch</code> runs and then line 11 runs: good; but if the <code>catch</code> itself threw (it cannot), <code>running</code> would stick; use <code>finally</code> for robustness (nit)
13	re-scheduling the same name overwrites the entry WITHOUT clearing the old interval: the old timer keeps firing forever, unreachable: leak and double-running

Line	Note
16–20	cancel clears and deletes: correct; no-op for unknown: clause 2
24	iterating <code>entries.keys()</code> while <code>cancel</code> deletes from the same Map: deleting the current key during Map iteration is safe in JS; continues to the next
25	after line 24, <code>entries</code> is EMPTY (every cancel deleted its entry), so <code>.some(...)</code> is always false and <code>shutdown</code> resolves immediately even if a run is in progress: clause 4 violated

Clauses

Clause	Status
1 no overlap	yes
2 cancel	yes; re-schedule leaks the old timer (13)
3 errors logged, schedule continues	yes
4 shutdown waits for in-progress	NO: entries are gone before the wait (24–25)

Findings

- Lines 24–25: `cancel` deletes entries, so the wait loop sees none; collect the entries first (`const live = [...this.entries.values()]`), cancel, then wait on `live.some(e => e.running)`.
- Line 13: scheduling an existing name must `cancel(name)` first, or the old interval leaks and both run.
- Line 11: use `finally` so `running` resets even if the catch path throws (nit).

Positives: overlap guard with no await between check and set; per-run try/catch with the name; cancel is idempotent; `undefined!` is justified by synchronous assignment.

Confidence 2 · Request changes.

Scoring yourself

Program	Must-find	Verdict
K	missing <code>return</code> after 422 (16), owner from body (19), temp file leaks (4)	1
L	<code>slice(-0)</code> (23), replay without try/catch (23)	3
M	shutdown waits on an emptied map (24–25), re-schedule leaks (13)	2